Name: Nabira Khan
Roll Number: 23K-0914

QUESTION#01

PART A

- Atomicity: because some operations executed (driver accepted, wallet deducted) but the transaction did not complete as one atomic unit.
- Durability: the confirmation was not recorded before the crash; the user never saw the committed state.
- Consistency: system moved to an inconsistent state: wallet deducted but ride not confirmed.

PART B

The violation occurred because the system did not execute the driver acceptance, wallet deduction, and ride confirmation within a single atomic transaction across its multiple services—lacking a distributed ACID transaction, two-phase commit, and write-ahead logging—so when the server crashed before commit, only partial updates were applied, breaking atomicity and consistency.

PART C

Use a single ACID transaction (or distributed 2-phase commit):

1. BEGIN TRANSACTION
2. Lock Driver_Status (write_lock)
3. Lock Wallet (write_lock)
4. Update Driver_Status = “Accepted”
5. Deduct Wallet Amount
6. Insert Ride Confirmation
7. COMMIT
8. Only after commit, return confirmation to the user.

Using strict 2PL:

- All write locks remain held until commit → no partial visibility → atomic, consistent, isolated, durable.

PART D

- Customers charged without rides → loss of trust
- Refund disputes → increased support cost
- Bad user reviews → market reputation loss
- Regulatory penalties (financial mismanagement)
- Drivers lose rides → revenue loss
- High churn → users switch to competing services.

Name: Nabira Khan
Roll Number: 23K-0914

QUESTION#02

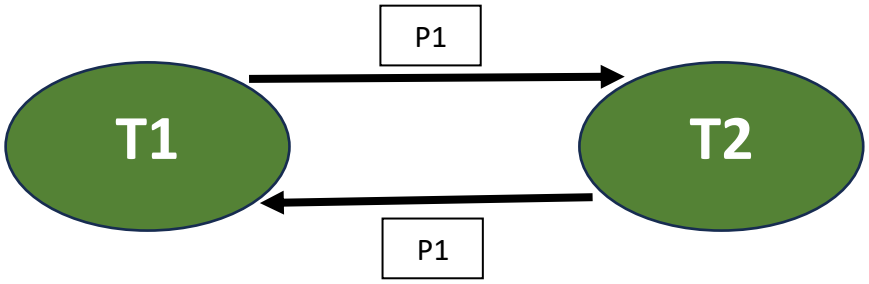
T1: Read(P.quantity) → Write(P.quantity - sold_units)
T2: Read(P.quantity) → Write(P.quantity + returned_units)

PART A

Classic lost update schedule:
1. R1(P1) // T1 reads quantity
2. R2(P1) // T2 reads quantity (concurrently)
3. W1(P1) // T1 writes quantity after sale
4. W2(P1) // T2 writes quantity after return

(Reads use the same original value, so one update overwrites the other's effect.)

PART B



PART C

No. Because the precedence graph has a cycle (T1 ↔ T2), the schedule is not conflict-serializable.

PART D

Two serial orders (both are safe); choose either. Example (T1 then T2):
1. R1(P1)
2. W1(P1)
3. R2(P1)
4. W2(P1)

Then other example is T2 then T1

PART E

- **Inventory miscounts:** overselling or stockouts.
- **Revenue loss:** missed sales (stock believed available when not) or returns not reflected.
- **Customer dissatisfaction:** canceled orders, delays, refunds.
- **Operational costs:** manual reconciliation, expedited shipments, lost supplier discounts.
- **Fraud / audit issues:** incorrect financial reporting.

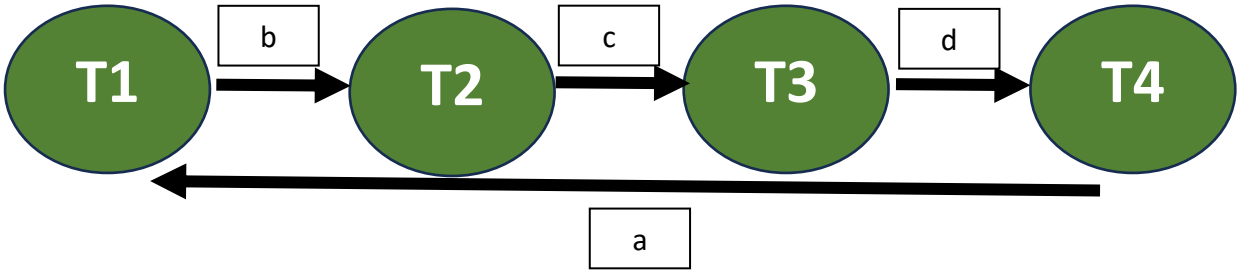
Name: Nabira Khan
Roll Number: 23K-0914

QUESTION#03

Transactions (operations summary):

- T1: r1(a); r1(b); w1(a)
- T2: r2(b); r2(c); w2(b)
- T3: r3(c); r3(d); w3(c)
- T4: r4(d); r4(a); w4(d)

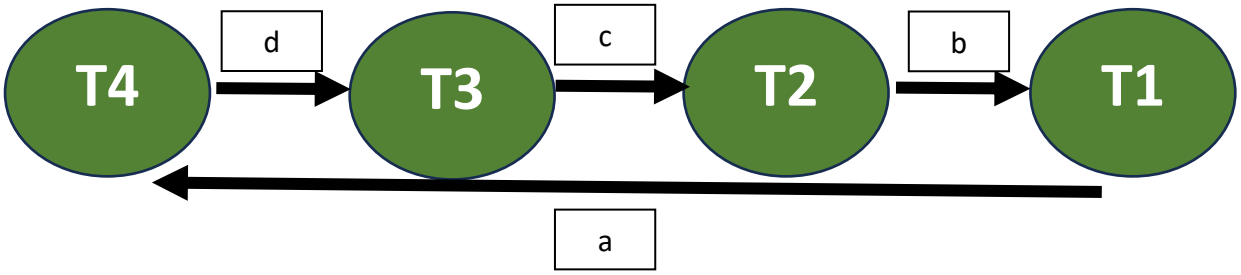
Schedule S1: S1: r1(a),r2(b),r3(c),r4(d),r1(b),r2(c),r3(d),r4(a),w2(b),w3(c),w4(d),w1(a)



Conclusion for S1:

- Precedence graph: cycle among T1,T2,T3,T4 (as above).
- Serializable? No — S1 is not conflict-serializable (cycle).

Schedule S2: S2:r4(d),r4(a),w4(d),r3(d),r3(c),w3(c),r2(b),r2(c),w2(b),r1(a),r1(b),w1(a)



Conclusion for S2:

- Precedence graph: T4 → T3 → T2 → T1 (acyclic).
- Serializable? Yes.
- Equivalent serial schedule(s): transactions in the order T4, T3, T2, T1 (that is the serial order consistent with the graph). That is the equivalent serial schedule.

QUESTION#04

PART 1

- **Lost Update Problem**: T1's discount (1700) is overwritten by T2's write (2800).
- **Write–Write conflict:** Both transactions write Total_Amount without isolation.
- **Non-serializable schedule**: Reads same old value, writes contradicting results.

PART 2

2800 (T2 overwrites T1's discount)

PART 3

Serial results:

- **T1 → T2**:
  - T1: 2000 − 300 = 1700
  - T2: 1700 + 800 = **2500**
- **T2 → T1**:
  - T2: 2000 + 800 = 2800
  - T1: 2800 − 300 = **2500**

Either serial order gives Rs 2,500. So the interleaved schedule produced incorrect final (2800) instead of the correct 2500.

QUESTION#05

(a) r1(X); w1(X); r3(X); r2(X); w3(X);

Conflicts:

- T1 → T3 (w1 before r3)
- T1 → T3 (w1 before w3)

Graph has no cycles → **Serializable**

Equivalent serial schedules:

T1 → T3 → T2

T1 executes before T3; T2 only reads so flexible.

(b) r1(X); r3(X); w3(X); w1(X); r2(X);

Conflicting operations on **X**:

- r1(X) → w3(X) → **T1 → T3**
- w3(X) → w1(X) → **T3 → T1**
- w3(X) → r2(X) → **T3 → T2**
- w1(X) → r2(X) → **T1 → T2**

(b) is NOT conflict-serializable

Reason: The graph contains a cycle **T1 → T3 → T1**, making serializability impossible.

(c) r3(X); r2(X); w3(X); r1(X); w1(X);

Conflicts:

- T3 → T1 (w3 before r1)
- T3 → T1 (w3 before w1)

No cycles → **Serializable**

Equivalent serial schedule:

T3 → T1 → T2

Name: Nabira Khan
Roll Number: 23K-0914

(d) r3(X); r2(X); r1(X); w3(X); w1(X);

Conflicting operations:

- r1(X) → w3(X) → **T1 → T3**
- r2(X) → w3(X) → **T2 → T3**
- r3(X) → w1(X) → **T3 → T1**

(d) is NOT conflict-serializable

Reason: The cycle **T1 → T3 → T1** makes the schedule non-serializable.